

Practical No: 07

Aim:

To create indexes and analyze query performance using the explain plan.

Theory:

Indexes play a crucial role in improving the speed and efficiency of database queries. In MongoDB, an index is a special data structure that stores a small portion of the dataset in an ordered form, making it faster for the database engine to locate specific documents. Without indexes, MongoDB performs a **collection scan**, meaning it checks every document in the collection to find matches. This becomes inefficient as data volume grows.

MongoDB supports various types of indexes such as single-field, compound, multikey, text, hashed, and geospatial indexes. However, this experiment focuses on the simplest and most commonly used type—**single-field index**. Creating an index using `createIndex()` does not modify any existing documents; instead, it creates a separate optimized structure that references the documents.

- **totalDocsExamined** — number of documents checked
- **totalKeysExamined** — number of index entries checked
- **executionTimeMillis** — how long the query took

Before creating an index, a query like `find({ dept: "CSE" })` typically results in a full collection scan, meaning `totalDocsExamined` equals the total number of documents in the collection. After creating an index on the `dept` field, the same query results in an index scan, where `totalKeysExamined` equals only the number of matching entries. This demonstrates how indexes reduce scan time and improve performance.

Commands:

Create a simple index on a single field:

```
db.students.createIndex({ dept: 1 })
db.students.find({ dept: "CSE" }).explain("executionStats")
```

Dropping Indexes:-

```
db.students.dropIndex("date_1")
```

Compound index

```
db.students.createIndex({ name: 1, age: -1 })
```

Unique index

```
db.students.createIndex({ email: 1 }, { unique: true })
```

Analyze Query Performance (Explain Plan)

```
db.users.find({ name: "John" }).explain("executionStats")
```

List all indexes in a collection:

```
db.students.getIndexes()
```

Check which index is being used for a query:

```
db.collection.explain("executionStats")
```

Text Index

```
db.users.createIndex({ description: "text" })
```

Query Planner Only

```
db.users.find({ age: { $gt: 25 } }).explain("queryPlanner")
```

All Plans Execution

```
db.users.find({ age: 30 }).explain("allPlansExecution")
```

Partial Index

```
db.users.createIndex(  
  { age: 1 },  
  { partialFilterExpression: { age: { $gt: 18 } } }  
)
```

Sparse Index

```
db.users.createIndex(  
  { phone: 1 },  
  { sparse: true }  
)
```

Background Index Creation

```
db.users.createIndex(  
  { city: 1 },  
  { background: true }  
)
```

Rebuild Indexes

```
db.users.reIndex()
```

Expected Output:

Index is created and explain output shows reduced scan time.

Viva Questions:

1. What is an index in MongoDB?

An index in MongoDB is a data structure that improves the speed of data retrieval operations by allowing the database to locate documents efficiently without scanning the entire collection.

2. What happens if a query is executed without an index?

MongoDB performs a **collection scan (COLLSCAN)**, where every document in the collection is examined, resulting in slower query performance.

3. What is the purpose of the explain() method?

The explain() method is used to analyze how MongoDB executes a query by displaying the query plan, execution time, and number of documents and index keys examined.

4. What is the difference between COLLSCAN and IXSCAN?

- **COLLSCAN:** MongoDB scans every document in the collection.
- **IXSCAN:** MongoDB scans only the relevant index entries, making the query faster.

Conclusion:

In this experiment, an index was successfully created on the dept field of the students collection using MongoDB's createIndex() command. The query execution was analyzed using the explain("executionStats") method. The results clearly showed that indexing reduced the number of documents scanned and improved query execution time by replacing a collection scan with an index scan.